

Branching

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

- | | |
|--|----------------------------------|
| 1. Introduction to Conditional Statement | 5. Nested if and else-if Ladders |
| 2. The if Statement | 6. The Conditional Operator |
| 3. The if-else Statement | 7. The switch Statement |
| 4. Case Study: The Leap Year Problem | 8. Problem Solving |

Introduction to Conditional Statement



The Flow of Execution

Sequential Execution

By default, C programs execute statements one after another, from top to bottom.

However, real life is full of decisions:

- **If** it rains, take an umbrella.
- **If** the traffic light is red, stop; else, go.
- **Switch** the TV channel based on the button pressed.

To handle these, we need **Conditional Statements**.

Conditional Statements

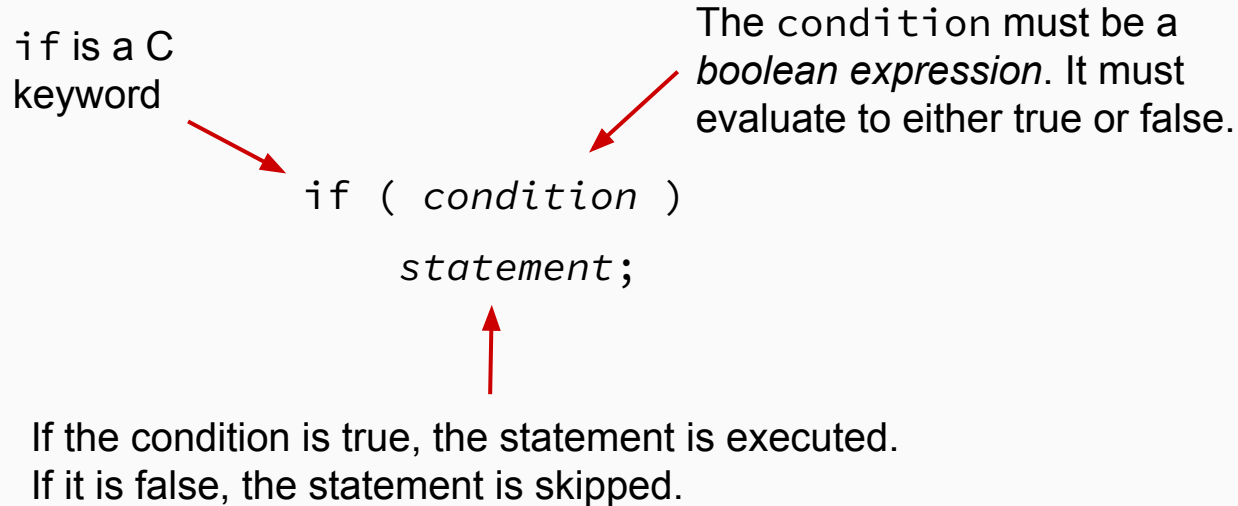
- A conditional statement lets us choose which statement will be executed next.
- Therefore they are sometimes called **selection statements**.
- Conditional statements give us the power to make basic decisions.
- The C conditional statements are the:
 - if statement
 - if-else statement
 - if-else if-else if-else ladder
 - switch-case statement
 - Conditional operator (?:)

The i f Statement



The if Statement

The if statement has the following syntax:



The if Statement (Example)

Selection structure:

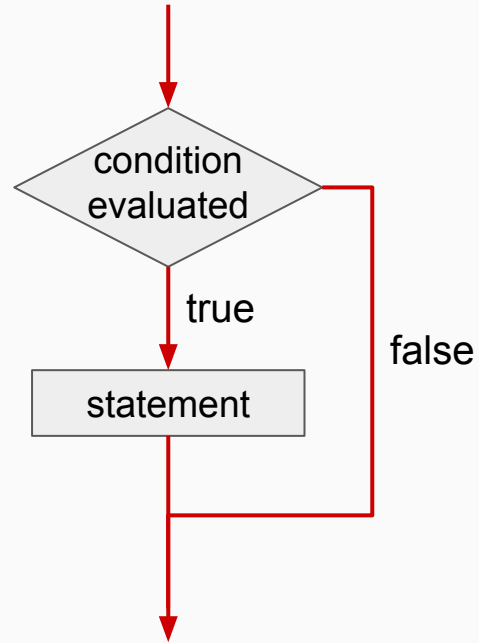
- Used to choose among alternative courses of action
- Pseudocode:

```
If student's mark at least 40
    Print "Passed"
```

Pseudocode statement in C:

```
#include<stdio.h>
int main() {
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);
    if (marks >= 40)
        printf("Passed\n");
    return 0;
}
```

Flowchart: if Statement



Relational Operators

A condition often uses one of C's equality operators or relational operators

<	less than	Higher Precedence
>	greater than	
<=	less than or equal to	
>=	greater than or equal to	

==	equal to	Lower Precedence
!=	not equal to	

Note the difference between the equality operator (==) and the assignment operator (=)

The if-else Statement



The if-else Statement

- An else clause can be added to an if statement to make an if-else statement

```
if ( condition )  
    statement1;  
else  
    statement2;
```

- If the condition is true, statement1 is executed;
if the condition is false, statement2 is executed
- One or the other will be executed, but not both

if-else statement analogy: Y-intersection



The if-else Statement (Example)

Selection structure:

- Used to choose among alternative courses of action
- Pseudocode:

```
If student's mark at least 40
    Print "Passed"
Otherwise
    Print "Failed"
```

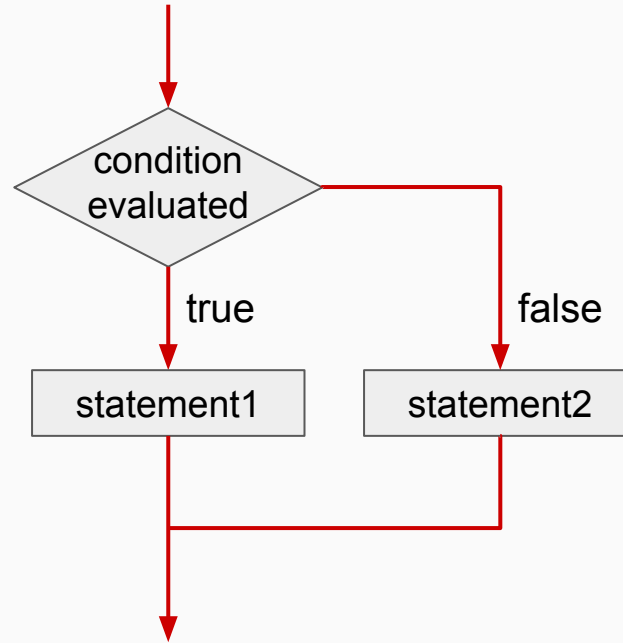
The if-else Statement (Example)

Selection structure:

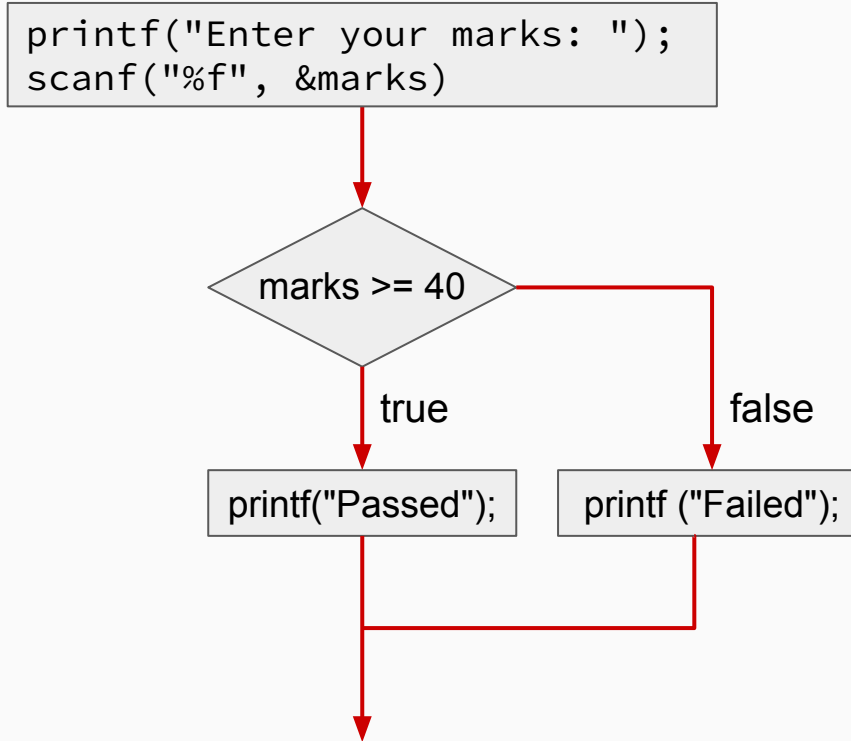
- Used to choose among alternative courses of action
- Pseudocode statement in C:

```
#include<stdio.h>
int main() {
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);
    if (marks >= 40)
        printf("Passed\n");
    else
        printf("Failed\n");
    return 0;
}
```

Flowchart: if-else Statement



Logic of previous example



Values on condition

- Zero (0) → False
- Anything nonzero → TRUE

```
if (40)
    printf("Hi\n");
else
    printf("Bye\n");
// Output: Hi
```

```
if (-40)
    printf("Hi\n");
else
    printf("Bye\n");
// Output: Hi
```

Relational Operators

- FALSE → Zero (0)
- TRUE → One (1)

```
a = 5;  
printf("%d", a > 5);  
// Output: 0
```

```
a = 5;  
printf("%d", a == 5);  
// Output: 1
```

Quiz

What is the output?

```
a = 6;  
b = 5;  
c = 2;  
if (a > b > c)  
    printf("Hi\n");  
else  
    printf("Bye\n");
```

- (a) Compile Error
- (b) Hi
- (c) Bye ✓

Logical NOT

- The logical NOT operation is also called logical negation or logical complement.
- If some condition a is true, then $!a$ is false; if a is false, then $!a$ is true.
- Logical expressions can be shown using a truth table.

cond	! cond
true	false
false	true

The if Statement (Example)

Selection structure:

- Used to choose among alternative courses of action
- Pseudocode:

```
If student's mark at least 40
    Print "Passed"
```

Pseudocode statement in C:

```
#include<stdio.h>
int main() {
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);
    if (marks >= 40)
        printf("Passed\n");
    return 0;
}
```

The if Statement (Example)

Selection structure:

- Used to choose among alternative courses of action
- Pseudocode:

```
If student's mark not smaller than 40
    Print "Passed"
```

Pseudocode statement in C:

```
#include<stdio.h>
int main() {
    int marks;
    printf("Enter your marks: ");
    scanf("%d", &marks);
    if (!(marks < 40))
        printf("Passed\n");
    return 0;
}
```

Block Statements

- Several statements can be grouped together into a block statement delimited by braces.
- A block statement can be used wherever a statement is called for in the C syntax rules.

```
if (b == 0)
{
    printf("divide by zero!!\n");
    errorCount++;
}
```

Block Statements

- In an if-else statement, the if portion, or the else portion, or both, could be block statements

```
if (b == 0) {  
    printf("divide by zero!!");  
    errorCount++;  
}  
else {  
    result = a/b;  
    printf("Result of division: %d", result);  
}
```

Example: Odd or Even

```
#include<stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);

    if (num % 2 == 0) {
        printf("%d is Even.\n", num);
    } else {
        printf("%d is Odd.\n", num);
    }
    return 0;
}
```

Example: Password Checker

```
int stored_pin = 1234;
int input_pin;

printf("Enter PIN: ");
scanf("%d", &input_pin);

if (input_pin == stored_pin) {
    printf("Access Granted.\n");
    printf("Welcome User.\n");
} else {
    printf("Access Denied.\n");
    printf("Alarm Triggered!\n");
}
```

Exercise

- Write down a program that will take two integers as input and will print the maximum of two.
- Write down a program that will take three integers as input and will print the maximum of three.
- Write down a program that will take three integers as input and will print the second largest of the three.

When we have multiple conditions

if-else extension

Vertical
Extension

Horizontal
Extension

When we have multiple conditions

if-else extension

*Vertical
Extension*

Horizontal
Extension

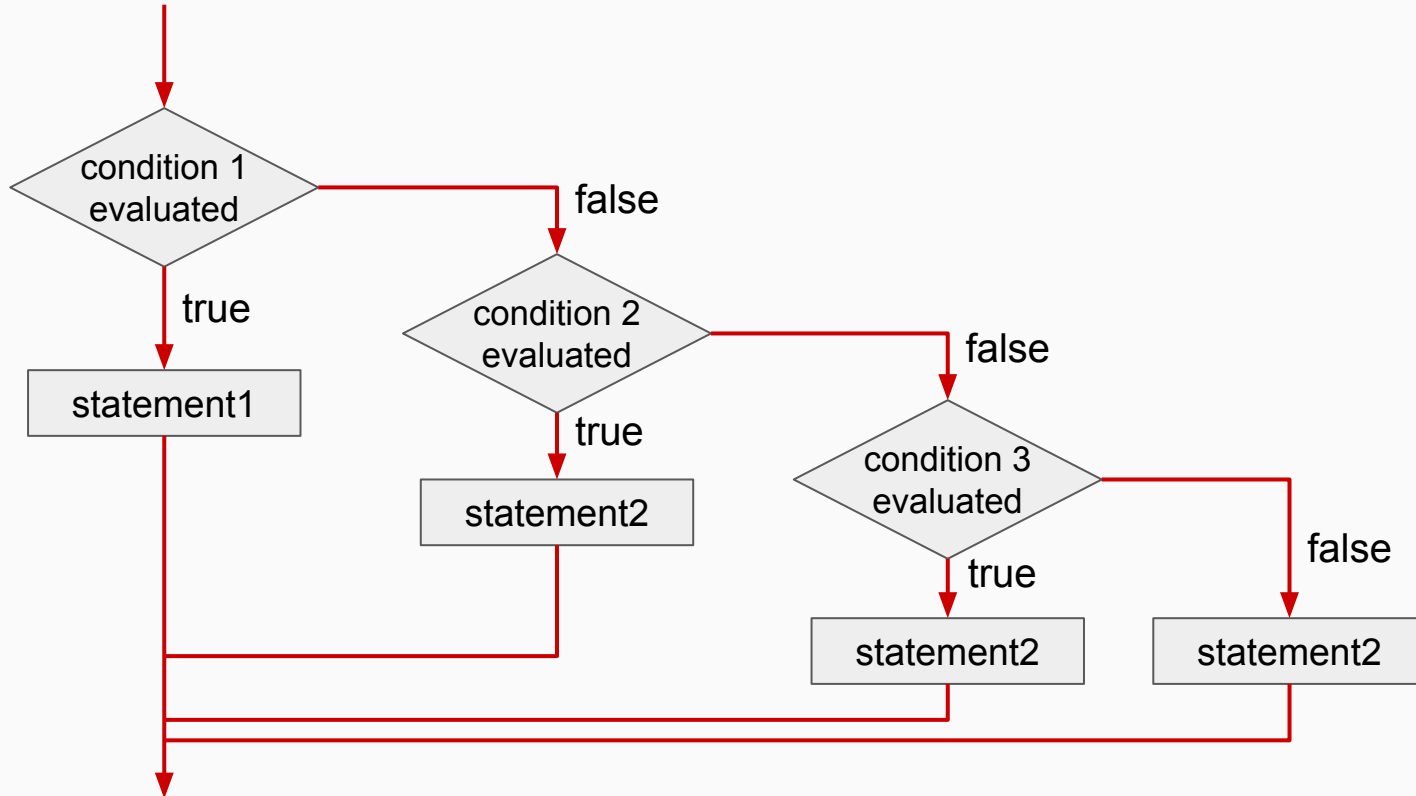
The if-else if-else if-else ladder

- If-else if- else if –else can be used to select from multiple choices:

```
if ( condition1 )  
    statement1;  
else if ( condition2 )  
    statement2;  
...  
...  
else if ( conditionk )  
    statementk;  
else  
    statement;
```

- If the *condition1* is true, *statement1* is executed; if *condition2* is true, *statement2* is executed; and so on

Flowchart: if-else if-else Statement



Example: Grading System

The following chart will be used for a quick grade conversion in C programming language course:

90-100	A
80-89	B
70-79	C
60-69	D
0-59	F

Write down a program that will take a student's mark as input and will convert it to the corresponding letter grade.

Example: Grading System

```
scanf("%d", &m);  
if (m >= 90)  
    g = 'A';  
else if (m >= 80)  
    g = 'B';  
else if (m >= 70)  
    g = 'C';  
else if (m >= 60)  
    g = 'D';  
else  
    g = 'F';  
printf("Mark = %d Grade= %c", m, g);
```

Example: Grading System

```
scanf("%d", &m);  
if (m >= 90)  
    g = 'A';  
else if (m >= 80)  
    g = 'B';  
else if (m >= 70)  
    g = 'C';  
else if (m >= 60)  
    g = 'D';  
else  
    g = 'F';  
printf("Mark = %d Grade= %c", m, g);
```

Example: Grading System

```
scanf("%d", &m);  
if (m >= 90)  
    g = 'A';  
if (m >= 80)  
    g = 'B';  
if (m >= 70)  
    g = 'C';  
if (m >= 60)  
    g = 'D';  
else  
    g = 'F';  
printf("Mark = %d Grade= %c", m, g);
```

Wrong!!

Example: Grading System

```
scanf("%d", &m);  
g = 'F';  
if (m >= 60)  
    g = 'D';  
if (m >= 70)  
    g = 'C';  
if (m >= 80)  
    g = 'B';  
if (m >= 90)  
    g = 'A';  
printf("Mark = %d Grade= %c", m, g);
```

**Correct but
inefficient!!**

When we have multiple conditions

if-else extension

Vertical
Extension

*Horizontal
Extension*

Combining multiple conditions: Logical Operators

- C defines the following logical operators:
 - `!` Logical NOT
 - `&&` Logical AND
 - `||` Logical OR
- Logical NOT is a unary operator (it operates on one operand). *We have already seen it before.*
- Logical AND and logical OR are binary operators (each operates on two operands).

Logical AND and Logical OR

- The logical AND expression

cond1 && cond2

is true if both a and b are true, and false otherwise.

- The logical OR expression

cond1 || cond2

is true if both a or b or both are true, and false otherwise.

Logical Operators

- A truth table shows all possible true-false combinations of the terms.
- Since `&&` and `||` each have two operands, there are four possible combinations of conditions a and b.

cond1	cond2	cond1 && cond2	cond1 cond2
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Example: Grading System

```
scanf("%d", &m);  
if (m >= 90 && m <= 100)  
    g = 'A';  
else if (m >= 70 && m < 80)  
    g = 'C';  
else if (m >= 80 && m < 90)  
    g = 'B';  
else if (m >= 60 && m < 70)  
    g = 'D';  
else  
    g = 'F';  
printf("Mark = %d Grade= %c", m, g);
```

Example: Grading System

```
scanf("%d", &m);  
if (m >= 90 && m <= 100)  
    g = 'A';  
if (m >= 70 && m < 80)  
    g = 'C';  
if (m >= 80 && m < 90)  
    g = 'B';  
if (m >= 60 && m < 70)  
    g = 'D';  
else  
    g = 'F';  
printf("Mark = %d Grade= %c", m, g);
```

Quiz

What is the output?

```
a = 6;  
b = 5;  
c = 2;  
if (a > b && b > c)  
    printf("Hi\n");  
else  
    printf("Bye\n");
```

- (a) Compile Error
- (b) Hi ✓
- (c) Bye

Case Study: The Leap Year Problem



Leap year explained

Problem: Determine if a given year is a Leap Year.

The Math Behind the Rules:

- A solar year is not exactly **365** days long.
- It takes approximately **365.2425** days for Earth to orbit the Sun.
- The extra fraction accumulates over time, so calendar adjustments are needed.
- Leap-year rules help keep the calendar synchronized with Earth's orbit.

Leap year explained

Leap year condition

1, 2, 3, 4, 5, 6, 7, 8, ..., 96, 100, 104, ..., 200, ..., 300, ..., 400, ..., 500, ..., 600, ..., 700, ..., 800, ..., 900, ..., 1000 ...

- Blue numbers leap year ➤ Multiple of 400
- Red numbers NOT leap year ➤ Multiple of 100 but not of 400
- Green numbers leap year ➤ Multiple of 4 but not of 100
- Black numbers NOT leap year ➤ Not divisible by 4 at all

Solution 1: Vertical Extension

Write down a program that will determine whether a year given as input is a leap year or not.

```
if (year % 400 == 0)
    printf("Leap year");
else if (year % 100 == 0)
    printf("Not Leap year");
else if (year % 4 == 0)
    printf("Leap year");
else
    printf("Not a leap year");
```

Solution 2: Horizontal Extension

Leap year condition

1, 2, 3, 4, 5, 6, 7, 8, ..., 96, 100, 104, ..., 200, ..., 300, ..., 400, ..., 500, ..., 600, ..., 700, ..., 800, ..., 900, ..., 1000 ...

- Blue numbers leap year ➤ Multiple of 400
- Green numbers leap year ➤ Multiple of 4 but not of 100

```
if ( ?? ) {  
    printf("Leap Year");  
} else {  
    printf("Common Year");  
}
```

Solution 2: Horizontal Extension

Leap year condition

1, 2, 3, 4, 5, 6, 7, 8, ..., 96, 100, 104, ..., 200, ..., 300, ..., 400, ..., 500, ..., 600, ..., 700, ..., 800, ..., 900, ..., 1000 ...

- Blue numbers leap year ➤ Multiple of 400
- Green numbers leap year ➤ Multiple of 4 but not of 100

```
if ( blue or green ) {  
    printf("Leap Year");  
} else {  
    printf("Common Year");  
}
```

Solution 2: Horizontal Extension

Leap year condition

1, 2, 3, 4, 5, 6, 7, 8, ..., 96, 100, 104, ..., 200, ..., 300, ..., 400, ..., 500, ..., 600, ..., 700, ..., 800, ..., 900, ..., 1000 ...

- Blue numbers leap year ➤ Multiple of 400
- Green numbers leap year ➤ Multiple of 4 but not of 100

```
if ((Multiple of 400) or (Multiple of 4 but not of 100)) {  
    printf("Leap Year");  
} else {  
    printf("Common Year");  
}
```

Solution 2: Horizontal Extension

Leap year condition

1, 2, 3, 4, 5, 6, 7, 8, ..., 96, 100, 104, ..., 200, ..., 300, ..., 400, ..., 500, ..., 600, ..., 700, ..., 800, ..., 900, ..., 1000

- Blue numbers leap year ➤ Multiple of 400
- Green numbers leap year ➤ Multiple of 4 but not of 100

```
if ((y%400 == 0) || ((y%4 == 0) && (y%100 != 0))) {  
    printf("Leap Year");  
} else {  
    printf("Common Year");  
}
```

Nested if and if-else Ladders



Nested if

Putting an if inside another if

Syntax

```
if (condition1) {  
    if (condition2) {  
        // Code executes if BOTH 1 and 2 are true  
    }  
}
```

Use this when the second decision depends on the first one passing

Example: Largest of Three Numbers

```
if (a > b) {  
    if (a > c) {  
        printf("A is largest");  
    } else {  
        printf("C is largest");  
    }  
} else {  
    // Here b >= a  
    if (b > c) {  
        printf("B is largest");  
    } else {  
        printf("C is largest");  
    }  
}
```

The Dangling else Problem

Which if does the else belong to?

Example

```
if (x > 0)
    if (y > 0)
        printf("Both Positive");
else
    printf("???");
```

Rule: An else attaches to the nearest preceding if that doesn't have an else.

Solution: Always use braces {} to be explicit!

The Art of Indentation

*“Any **fool** can write code that a computer can understand. Good **programmers** write code that humans can understand.”*

- Martin Fowler

What is Indentation?

- Adding spaces or tabs before a statement.
- Used to show which statements belong to a block (if, else, etc.).
- Makes code easier to read and understand.

Example:

```
if (x > 0) {  
    printf("Positive\n");  
    printf("Valid Input\n");  
}
```

Good vs. Bad Indentation

Bad Code (Flat)

```
if(x>0){  
if(y>0){  
printf("Q1");  
}else{  
printf("Q4");  
}  
}else{  
printf("Left");  
}
```

Hard to trace.

Good Code (Indented)

```
if (x > 0) {  
    if (y > 0) {  
        printf("Q1");  
    } else {  
        printf("Q4");  
    }  
} else {  
    printf("Left");  
}
```

Structure is clear.

The Art of Indentation

Why Indent?

- **Readability:** Clearly shows logical structure.
- **Hierarchy:** Visualizes which statements belong to which block.
- **Debugging:** Helps spot missing braces } or logical errors, especially in nested logic.

Note: In C, indentation is ignored by the compiler, but it is critical for you (and whoever is reading your code)!

The Conditional Operator



Conditional Operator (? :)

C has a *conditional operator* that uses a boolean condition to determine which of two expressions is evaluated.

Syntax

```
condition ? expression1 : expression2;
```

- If condition is **True**, expression1 is evaluated.
- If condition is **False**, expression2 is evaluated.

The value of the entire conditional operator is the value of the selected expression.

Conditional Operator (? :)

The conditional operator is similar to an if-else statement, except that it is an expression that returns a value.

Example

```
larger = ((num1 > num2) ? num1 : num2);
```

- If num1 is greater than num2, then num1 is assigned to larger;
- Otherwise, num2 is assigned to larger.

The conditional operator is *ternary* because it requires three operands.

Example: Ternary vs If-Else

Task: Find the maximum of a and b.

Using if-else:

```
if (a > b)
    max = a;
else
    max = b;
```

Using Ternary:

```
max = (a > b) ? a : b;
```

Example: Absolute Value

Writing a concise absolute value macro or statement.

```
int num = -10;  
int abs_val;  
  
abs_val = (num < 0) ? -num : num;  
  
printf("Absolute value: %d\n", abs_val);  
// Output : 10
```

The switch Statement

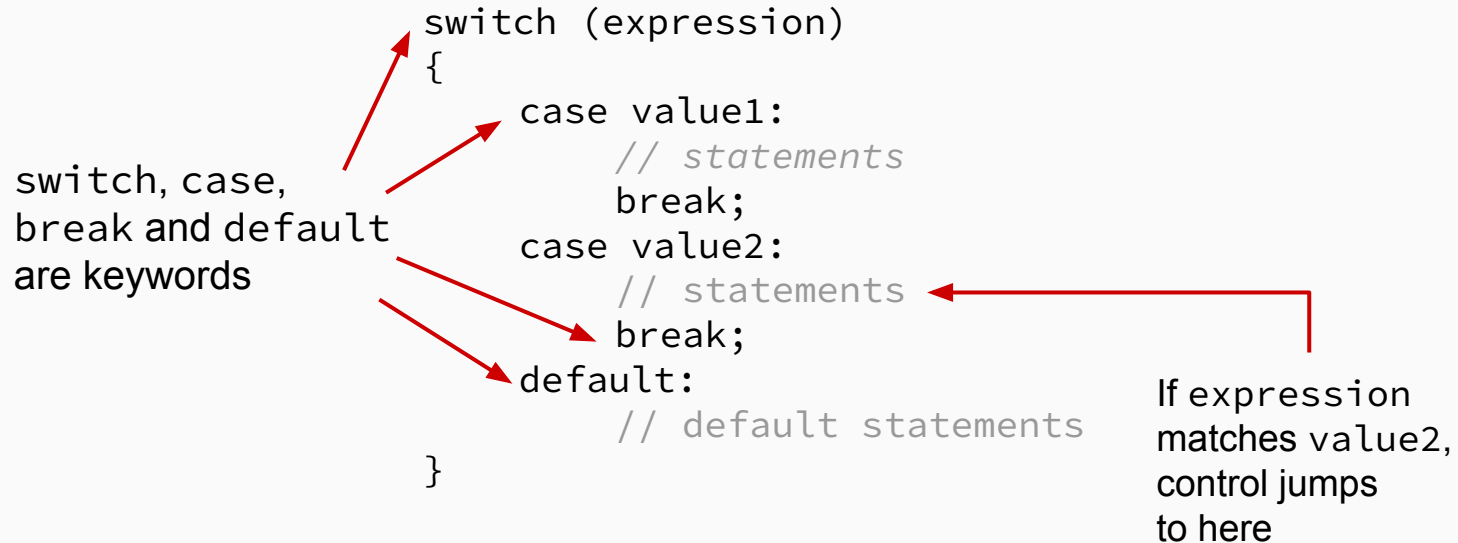


The switch Statement

- The *switch* statement provides another way to decide which statement to execute next.
- The *switch* statement evaluates an expression, then attempts to match the result to one of several possible cases.
- Each case contains a value and a list of statements.
- The flow of control transfers to statement associated with the first case value that matches.

The switch Statement

The general syntax of a switch statement is:



switch Rules

1. The **expression** must evaluate to an **integer** or **character**. (No float, no string).
2. **case values** must be constant literals (e.g., 5, 'A'). You cannot use variables like `case x:`.
3. **break** prevents “fall-through”. Without it, execution continues to the next case.
4. **default** is optional (like the final `else`).

The switch Statement: Example

Write down a program using switch-case structure that will take an integer as input and will determine whether the number is odd or even.

```
switch (n % 2)
{
    case 0:
        printf("It is Even");
        break;
    case 1:
        printf("It is ODD");
        break;
}
```

The switch Statement: Example 1

Write down a program using switch structure that will take an integer as input and will determine whether the number is multiple of 3 or not.

```
switch (n % 3)
{
    case 0:
        printf("It is Multiple of 3");
        break;
    default:
        printf("No it's not");
        break;
}
```

The switch Statement: Example 1

Write down a program using switch structure that will take an integer as input and will determine whether the number is multiple of 3 or not.

```
switch (n % 3)
{
    case 0:
        printf("It is Multiple of 3");
        break;
    default:
        printf("No it's not");
        break;
}
```

If n is multiple of 3, both will be printed.

The switch Statement: Example 2

Write down a program that will take a character as input and will determine whether it is a vowel or consonant.

```
scanf("%c", &ch);
switch (ch)
{
    case 'a':printf("It is Vowel");
              break;
    case 'e':printf("It is Vowel");
              break;
    .....
    case 'u':printf("It is Vowel");
              break;
    default: printf("It is consonant");
}
}
```

The switch Statement: Fall-through

Write down a program that will take a character as input and will determine whether it is a vowel or consonant.

```
scanf("%c", &ch);
switch (ch)
{
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u': printf("It is Vowel");
              break;
    default: printf("It is consonant");
}
```

Using *fall-through* feature to group cases.

The switch Statement: Example 3

Write down a program using switch-case structure that will take two integers as input and will determine the maximum of two.

```
switch (a > b)
{
    case 0:
        max = b;
        break;
    case 1:
        max = a;
        break;
}
```

Example 4: Simple Menu

```
int choice ;
printf("1. Start Game\n2. Load Game\n3. Exit\n");
scanf("%d", &choice) ;

switch (choice) {
    case 1:
        printf("Starting...\n");
        break;
    case 2:
        printf("Loading...\n");
        break;
    case 3:
        printf("Goodbye!\n");
        break;
    default:
        printf("Invalid input.\n");
}
```

switch vs if-else

switch Case

- Works only with `int` / `char`.
- Checks equality only (`==`).
- Good for menus, state machines.

if-else Ladder

- Works with all types (`float`, etc.).
- Checks any condition (`>`, `<`, `&&`, etc.).
- Good for ranges and complex logic.

Problem Solving



A Common Mistake

What is the output of the following code snippet?

```
int x = 10;
if (x = 5) {
    printf("True: %d", x);
} else {
    printf("False: %d", x);
}
```

Output:

True: 5

Reason: The code uses = (assignment), not == (equality).

1. x is assigned 5.
2. The expression evaluates to 5 (non-zero), which is True.
3. The body executes.

Example: Traffic Light System

Write a switch-case program that prints instructions based on character input:

- 'R' or 'r' → "Stop"
- 'Y' or 'y' → "Get Ready"
- 'G' or 'g' → "Go"
- Other → "Traffic Signal Error"

Solution: Traffic Light System Solution

```
char color ;
scanf (" %c", &color);

switch (color) {
    case 'R':
    case 'r':
        printf("Stop\n");
        break;
    case 'Y':
    case 'y':
        printf("Get Ready\n");
        break ;
    case 'G':
    case 'g':
        printf("Go\n");
        break;
    default:
        printf("Traffic Signal Error\n");
}
```

Example: Grading System

The following chart will be used for a quick grade conversion in C programming language course:

90-100	A
80-89	B
70-79	C
60-69	D
0-59	F

Write down a program that will take a student's mark as input and will convert it to the corresponding letter grade. Solve this using:

1. `if-else if-else` ladder.
2. `switch-case` statement.

Solution A: Grading with if-else

The natural choice for **ranges**.

```
scanf("%d", &m);  
if (m >= 90)  
    printf("Grade A");  
else if (m >= 80)  
    printf("Grade B");  
else if (m >= 70)  
    printf("Grade C");  
else if (m >= 60)  
    printf("Grade D");  
else  
    printf("Grade F");
```

Solution B: Grading with switch

Trick: Divide marks by 10 to get the grouping prefix!

```
switch (marks / 10) {  
    case 10:  
    case 9:  
        printf("Grade A"); break;  
    case 8:  
        printf("Grade B"); break;  
    case 7:  
        printf("Grade C"); break;  
    case 6:  
        printf("Grade D"); break;  
    default:  
        printf("Grade F");  
}
```

List of all Keywords in C Language

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Summary

1. Use `if` for single conditional execution.
2. Use `if-else` for “this or that” logic.
3. Use `else-if` ladders for ranges and complex multi-condition logic.
4. Use `switch` for checking discrete values (menus, commands).
5. Use `?:` (ternary operator) for simple assignments based on conditions.
6. Always verify indentation and braces to avoid logical errors.

Acknowledgement

- Dr. Ashikur Rahman, Professor, CSE, BUET
- Mohammad Sadat Hossain, Lecturer, CSE, BUET